COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE-010	**SECTION:**	CPE21S2
DATE PERFORMED:	7/24/26	**DATE SUBMITTED:**	
NAME:	SUBALISID, JAPPETH ROYCE B.	**INSTRUCTOR:**	ENGR. JIMLORD QUEJADO

ACTIVITY NO. ___ [ACTIVITY NAME]

1. PROCEDURE OUTPUT

Screenshot

```cpp
#include<iostream>
#include<utility>
class Node{
  public:
  char data;
  Node *next;
  };

void traversal(Node* head) {
  while(head != nullptr) {
    std::cout << head->data;
    head = head->next;
  }
}

int main(){
//step 1
    Node *head = NULL;
    Node *second = NULL;
    Node *third = NULL;
    Node *fourth = NULL;
    Node *fifth = NULL;
    Node *last = NULL;
    //step 2
    head = new Node;
    second = new Node;
    third = new Node;
    fourth = new Node;
    fifth = new Node;
    last = new Node;
    //step 3
    head->data = 'C';
    head->next = second;
    second->data = 'P';
    second->next = third;
    third->data = 'E';
    third->next = fourth;
    fourth->data = '0';
    fourth->next = fifth;
    fifth->data = '1';
    fifth->next = last;
    //step 4
    last->data = '0';
    last->next = nullptr;
    traversal(head);

    return 0;
}

```

Discussion	This is a basic code of a singly linked list where a node class is made and data are assigned to it accordingly. Then the program calls a traverse function where it prints all the contents of each node in order of the list.

Operation	**Screenshot**

Traversal	`//traversal`
	`std::cout<<"\nTesting the DLL Traversal";`
	`dllTraverse(head);`

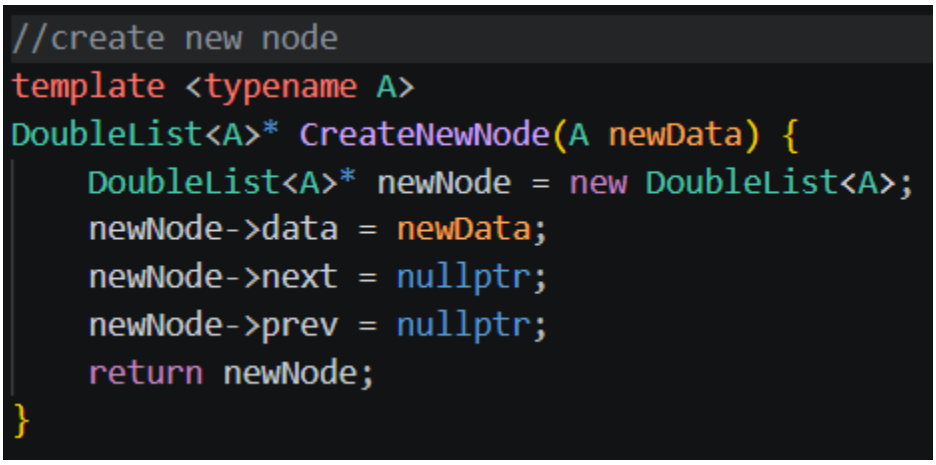
Insertion at Head	`//insert head`
	`dllInsertHead('X', head);`
	`dllTraverse(head);`

Insertion at tail	`//insert at tail`
	`dllInsertEnd('Y', head);`
	`dllTraverse(head);`

Insertion at middle	`//gen insert`
	`dllGeneralInsert('O', head->next);`
	`dllTraverse(head);`

Deletion of a Node	`//deletes`
	`dllDeleteNode('O', head);`
	`dllTraverse(head);`
	
	`dllDeleteNode('Y', head);`
	`dllTraverse(head);`
	
	`dllDeleteNode('X', head);`
	`dllTraverse(head);`

Table 3-2. Code for the List Operations

a.	Source Code	
	Console	No terminal output

```
//create new node
template <typename A>
DoubleList<A>* CreateNewNode(A newData) {
    DoubleList<A>* newNode = new DoubleList<A>;
    newNode->data = newData;
    newNode->next = nullptr;
    newNode->prev = nullptr;
    return newNode;
}
```

b. Source Code

```
//Traverse print
template <typename A>
void dllTraverse(DoubleList<A>* currentNode){
    //temp variable to store the last node visited
    DoubleList<A> *tail;

    //check if the list is empty
    if(currentNode == nullptr){
        std::cout<<"The list is empty"<<std::endl;
        return;
    }
    //move forward
    std::cout << "\nForward\n";
    while(currentNode != nullptr){
        std::cout<< currentNode->data << " ";
        tail = currentNode;
        currentNode = currentNode->next;
    }

    std::cout << "\nBackward\n";

    while(tail != nullptr){
        std::cout<< tail->data << " ";
        tail = tail->prev;
    }
    std::cout << std::endl;
}
```

Console

```
Forward
C P E
Backward
E P C
```

c. Source Code

```
//insert at head
template <typename A>
void dllInsertHead(A newData, DoubleList<A>*& currentHead) {
    //Announce
    std::cout<<"\nInserting '" << newData << "' at the head";
    //create new node
    DoubleList<A>* newNode = CreateNewNode(newData);
    //newnode point to current head
    newNode->next = currentHead;
    //current head point prev to newnode
    currentHead->prev = newNode;
    //update the pointer head
    currentHead = newNode;
}
```

Console

```
Inserting 'X' at the head
Forward
X C P E
Backward
E P C X
```

*Called traverse() after inserting

d. Source Code

```
//Insert at tail
template <typename A>
void dllInsertEnd(A newData, DoubleList<A>* currentNode) {
    //Announce
    std::cout<<"\nInserting '" << newData << "' at the end";
    //create new node
    DoubleList<A>* newNode = CreateNewNode(newData);
    //find tail
    while(currentNode->next != nullptr) {
        currentNode = currentNode->next;
    }
    //connect last node to new node
    currentNode->next = newNode;

    newNode->prev = currentNode;
}
```

	Console	``` Inserting 'Y' at the end Forward X C P E Y Backward Y E P C X ``` *Called traverse() after inserting

e.	Source Code	```
		//General Insert
		template <typename A>
		void dllGeneralInsert(A newData, DoubleList<A>* prevNode) {
		    //Announce
		    std::cout<<"\nInserting '" << newData << "' after " << prevNode->data;
		    //temp for next node
		    DoubleList<A>* nextNode;
		    //Check if prev node is null
		    if(prevNode == nullptr) {
		        std::cout << "Previous value cannot be null...\n";
		        return;
		    }
		    //create newNode
		    DoubleList<A>* newNode = CreateNewNode(newData);
		    //temp next node
		    nextNode = prevNode->next;
		
		    //point prevnode to newnode
		    prevNode->next = newNode;
		    newNode->prev = prevNode;
		
		    //point newnode to next and back
		    newNode->next = nextNode;
		    nextNode->prev = newNode;
		}
		``

	Console	``` Inserting 'O' after C Forward X C O P E Y Backward Y E P O C X ``` *Called traverse() after inserting

f.	Source Code	

```cpp
//Delete Node
template <typename A>
void dllDeleteNode(A findData, DoubleList<A>*& start) {
    //Announce
    std::cout<<"\nDeleting '" << findData <<'\'';
    //if theres nothing to delete
    if(start == nullptr) {
        std::cout << "\nNo data to delete\n";
        return;
    }

    DoubleList<A>* currNode = start;

    //data doesnt exist or found
    while(currNode != nullptr && currNode->data != findData) {
        currNode = currNode->next;
    }

    //data not found
    if(currNode == nullptr) {
        std::cout << "Data was not found...\n";
        return;
    }

    //data found at head
    if(currNode->prev == nullptr) {
        start = currNode->next;
        if (start != nullptr) {
          start->prev = nullptr;
        }
    //at tail
    }else if(currNode->next == nullptr) {
        currNode->prev->next = nullptr;
    //at middle
    }else {
        currNode->prev->next = currNode->next;
        currNode->next->prev = currNode->prev;
    }

    delete currNode;
}

```

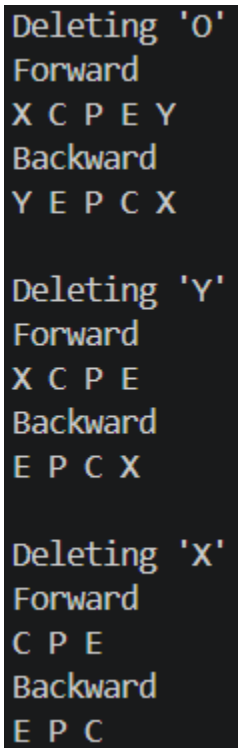
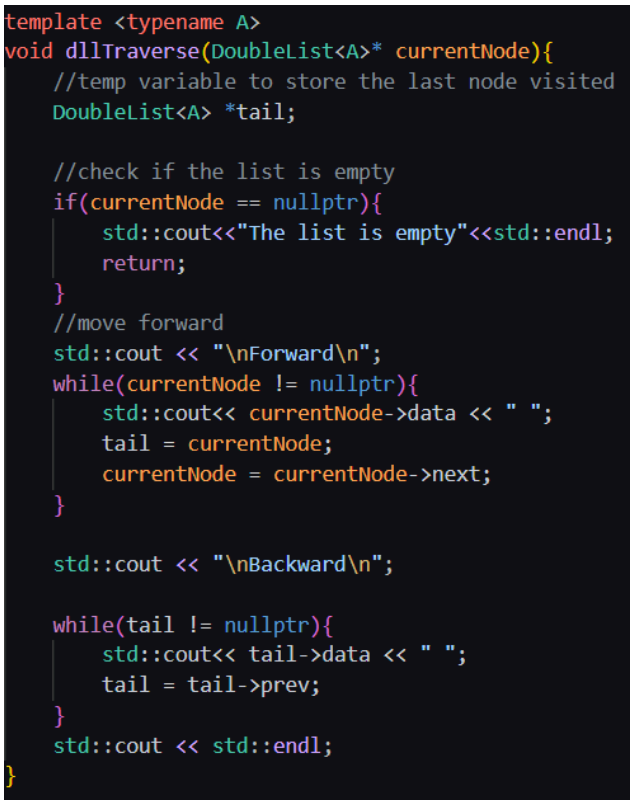
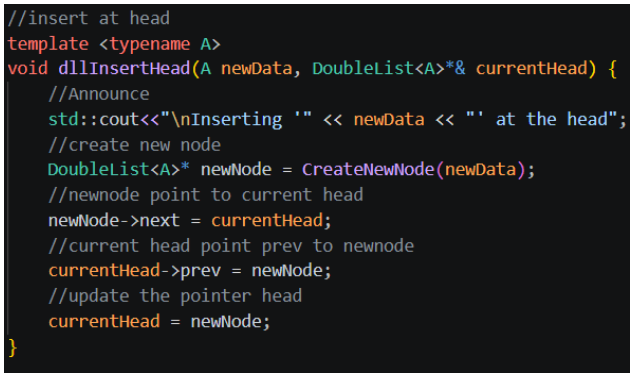
	Console	 *Called traverse() after deleting

Table 3-3. Code and Analysis for Doubly Linked Lists

Screenshot(s)	Analysis


``
template <typename A>
void dllTraverse(DoubleList<A>* currentNode){
    //temp variable to store the last node visited
    DoubleList<A> *tail;
 	
    //check if the list is empty
    if(currentNode == nullptr){
        std::cout<<"The list is empty"<<std::endl;
        return;
    }
    //move forward
    std::cout << "\nForward\n";
    while(currentNode != nullptr){
        std::cout<< currentNode->data << " ";
        tail = currentNode;
        currentNode = currentNode->next;
    }
 	
    std::cout << "\nBackward\n";
 	
    while(tail != nullptr){
        std::cout<< tail->data << " ";
        tail = tail->prev;
    }
    std::cout << std::endl;
}
``
	• The function utilizes the same operations as in the single list traversal, just that there is additional steps to print the reverse order
	• In printing the reverse order, we have to use a temp pointer to get a reference to the last looked node. We use that temp as the argument for the prints and move backwards from there


``
//insert at head
template <typename A>
void dllInsertHead(A newData, DoubleList<A>*& currentHead) {
    //Announce
    std::cout<<"\nInserting '" << newData << "' at the head";
    //create new node
    DoubleList<A>* newNode = CreateNewNode(newData);
    //newnode point to current head
    newNode->next = currentHead;
    //current head point prev to newnode
    currentHead->prev = newNode;
    //update the pointer head
    currentHead = newNode;
}
``
	• Inserting at head on a doubly linked list is the same as a singly linked list, only that we have to point one of the previous head's pointer back to the new head-node.

```cpp
//General Insert
template <typename A>
void dllGeneralInsert(A newData, DoubleList<A>* prevNode) {
    //Announce
    std::cout<<"\nInserting '" << newData << "' after " << prevNode->data;
    //temp for next node
    DoubleList<A>* nextNode;
    //Check if prev node is null
    if(prevNode == nullptr) {
        std::cout << "Previous value cannot be null...\n";
        return;
    }
    //create newNode
    DoubleList<A>* newNode = CreateNewNode(newData);
    //temp next node
    nextNode = prevNode->next;

    //point prevnode to newnode
    prevNode->next = newNode;
    newNode->prev = prevNode;

    //point newnode to next and back
    newNode->next = nextNode;
    nextNode->prev = newNode;
}
```

- General insert is a bit difficult to do, but I utilized a temporary pointer to store the next node's address since it will be lost if I didn't. Then I just pointed the previous node's 'next' to the new node and back. Then using the temp, I pointed the new node's 'next' to that and back to restore the connections.

```cpp
//Insert at tail
template <typename A>
void dllInsertEnd(A newData, DoubleList<A>* currentNode) {
    //Announce
    std::cout<<"\nInserting '" << newData << "' at the end";
    //create new node
    DoubleList<A>* newNode = CreateNewNode(newData);
    //find tail
    while(currentNode->next != nullptr) {
        currentNode = currentNode->next;
    }
    //connect last node to new node
    currentNode->next = newNode;

    newNode->prev = currentNode;
}
```

- Inserting at end was also simple since we only do the same thing as insert head, only that we have to find the tail before doing the operations

```
//Delete Node
template <typename A>
void dllDeleteNode(A findData, DoubleList<A>*& start) {
    //Announce
    std::cout<<"\nDeleting '" << findData <<'\'';
    //if theres nothing to delete
    if(start == nullptr) {
        std::cout << "\nNo data to delete\n";
        return;
    }

    DoubleList<A>* currNode = start;

    //data doesnt exist or found
    while(currNode != nullptr && currNode->data != findData) {
        currNode = currNode->next;
    }

    //data not found
    if(currNode == nullptr) {
        std::cout << "Data was not found...\n";
        return;
    }

    //data found at head
    if(currNode->prev == nullptr) {
        start = currNode->next;
        if (start != nullptr) {
          start->prev = nullptr;
        }
    //at tail
    }else if(currNode->next == nullptr) {
        currNode->prev->next = nullptr;
    //at middle
    }else {
        currNode->prev->next = currNode->next;
        currNode->next->prev = currNode->prev;
    }

    delete currNode;
}

```

- Deleting a node is also similar to the singly linked list. We create a temporary node and find the similar data, then assign the adjacent nodes to each other then delete the found node.
- If we don't find anything, we print it, and if we find it, we delete it.

Table 3-4. Modified Operations for Doubly Linked Lists

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B: Solve given problems utilizing linked lists in C++
Problem Title: Implementing a Song Playlist using Linked List
Source: Packt Publishing
Problem Description:
In this activity, we'll look at some applications for which a singly linked list is not enough or not convenient. We will build a tweaked version that fits the application. We often encounter cases where we have to customize default implementations, such as when looping songs in a music player or in games where multiple players take a turn one by one in a circle.

These applications have one common property – we traverse the elements of the sequence in a circular fashion. Thus, the node after the last node will be the first node while traversing the list. This is called a circular linked list. We'll take the use case of a music player. It should have following functions supported:

Create a playlist using multiple songs.

- Add songs to the playlist.

```
//initialize songs
DoubleList<std::string>* song1 = AddSong<std::string>("Kenshi Yonezu - Campanella");
DoubleList<std::string>* song2 = AddSong<std::string>("Kenshi Yonezu - Kanden");
DoubleList<std::string>* song3 = AddSong<std::string>("Kenshi Yonezu - Lemon");
DoubleList<std::string>* song4 = AddSong<std::string>("Kenshi Yonezu - Machigai Sagashi");
DoubleList<std::string>* song5 = AddSong<std::string>("Kenshi Yonezu - Canary");
DoubleList<std::string>* song6 = AddSong<std::string>("Kenshi Yonezu - Uma to Shika");
```

```
Songs:
Kenshi Yonezu - Campanella
Kenshi Yonezu - Kanden
Kenshi Yonezu - Lemon
Kenshi Yonezu - Machigai Sagashi
Kenshi Yonezu - Canary
Kenshi Yonezu - Uma to Shika
```

```
//adding songs
InsertStart<std::string>("Kenshi Yonezu - Karasu", song1);
InsertEnd<std::string>("Kenshi Yonezu - Yodaka", song1);
InsertAfter<std::string>("Kenshi Yonezu - Jane Doe", song3);
PlayList(song1);
```

```
Inserting 'Kenshi Yonezu - Karasu' at the head
Inserting 'Kenshi Yonezu - Yodaka' at the end
Inserting 'Kenshi Yonezu - Jane Doe' after Kenshi Yonezu - Lemon
Songs:
Kenshi Yonezu - Karasu
Kenshi Yonezu - Campanella
Kenshi Yonezu - Kanden
Kenshi Yonezu - Lemon
Kenshi Yonezu - Jane Doe
Kenshi Yonezu - Machigai Sagashi
Kenshi Yonezu - Canary
Kenshi Yonezu - Uma to Shika
Kenshi Yonezu - Yodaka
```

- Remove a song from the playlist.

```
//deleting song
DeleteSong<std::string>("Kenshi Yonezu - Kanden", song1);
PlayList(song1);
```

```
Deleting 'Kenshi Yonezu - Kanden'
Songs:
Kenshi Yonezu - Karasu
Kenshi Yonezu - Campanella
Kenshi Yonezu - Lemon
Kenshi Yonezu - Jane Doe
Kenshi Yonezu - Machigai Sagashi
Kenshi Yonezu - Canary
Kenshi Yonezu - Uma to Shika
Kenshi Yonezu - Yodaka
```

- Play songs in a loop (for this activity, we will print all the songs once).

```
//looping list
template <typename A>
void LoopTimes(int n, DoubleList<A>* start) {
    std::cout << "\nLooping " << n << " times.\n";
    DoubleList<A>* temp = start;
    for(int i = 0; i < n; i++){
        while(temp != nullptr) {
            std::cout << temp->data << '\n';
            temp = temp->next;
        }
        temp = start; //making sure it loops back to start
        std::cout << temp->data << '\n';
        temp = temp->next;
    }
}
```

```
//looping songs n times
LoopTimes(1, song1);
```

```
Looping 1 times.
Kenshi Yonezu - Karasu
Kenshi Yonezu - Campanella
Kenshi Yonezu - Lemon
Kenshi Yonezu - Jane Doe
Kenshi Yonezu - Machigai Sagashi
Kenshi Yonezu - Canary
Kenshi Yonezu - Uma to Shika
Kenshi Yonezu - Yodaka
Kenshi Yonezu - Karasu
```

```
//looping songs n times
LoopTimes(2, song1);
```

```
Looping 2 times.
Kenshi Yonezu - Karasu
Kenshi Yonezu - Campanella
Kenshi Yonezu - Lemon
Kenshi Yonezu - Jane Doe
Kenshi Yonezu - Machigai Sagashi
Kenshi Yonezu - Canary
Kenshi Yonezu - Uma to Shika
Kenshi Yonezu - Yodaka
Kenshi Yonezu - Karasu
Kenshi Yonezu - Campanella
Kenshi Yonezu - Lemon
Kenshi Yonezu - Jane Doe
Kenshi Yonezu - Machigai Sagashi
Kenshi Yonezu - Canary
Kenshi Yonezu - Uma to Shika
Kenshi Yonezu - Yodaka
Kenshi Yonezu - Karasu
```

Here are the steps to solve the problem:

- Design the basic structure that supports circular data representation.
- After that, implement the insert and delete functions in the structure.
- Implement a function for traversing the playlist.

The driver function should allow for common operations on a playlist such as: next, previous, play all songs, insert and remove.

```
//play from end
template <typename A>
void ReversePlay(DoubleList<A>* start) {
    std::cout << "\nPlaying from the end\n";
    DoubleList<A>* temp = start;
    while(temp->next != nullptr) {
        temp = temp->next;
    }
    while(temp != nullptr) {
        std::cout << temp->data << '\n';
        temp = temp->prev;
    }
}
```

```
//play from end
ReversePlay(song1);
```

Conclusion:

This supplemental activity helped me understand how to use and make functions of a doubly linked list. I also learned how to manipulate the order of reading a linked list whether forward or backward and how to loop in a list. When I saw the problem given, I immediately thought of linking the end and beginning with a pointer, but I found it to be easier if i just played in different modes—from the start, back from the end, and looping. This, to me, represents a playlist more than if the playlist automatically looped on its own. The header file is mostly the same with the doubly part of the activity, and I just edited the function names and added two more for the playing modes.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used:

Gemini

Purpose of Use:

Help in finding bugs and simple fixes

Extent of Use:

The majority of the code was generated by me and the AI tool was only utilized when I got stuck and cannot

find the bugs.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.